



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

PROYECTO DE CARRERA: INGENIERÍA INFORMÁTICA

UNIDAD CURRICULAR : LENGUAJES Y COMPILADORES

SEMESTRE VII – 2026 – I

ANÁLISIS SINTÁCTICO

Profesor:

Marquez, Felix

Estudiantes:

Frontado, Keiner C.I. 32.059.780

Martínez, Giovanni C.I. 31.782.337

Morales, Edgar C.I. 30.631.492.

Ortega, Mansour C.I. 28.785.297

Ciudad Guayana, julio 2026

Introducción

El desarrollo de compiladores e intérpretes modernos exige una rigurosa fundamentación conceptual sobre cómo se transforma el código fuente en estructuras ejecutables y procesables por la máquina. Si bien el análisis léxico constituye el primer filtro para clasificar el flujo plano de caracteres en componentes con significado (tokens), es en la fase de análisis sintáctico (parsing) donde se determina la validez estructural del programa de acuerdo con las reglas de una Gramática Libre de Contexto (GLC).

Examinando la naturaleza del Árbol de Sintaxis Abstracta (AST) como una representación jerárquica desacoplada de elementos accesorios o puramente decorativos de la sintaxis concreta. Se analiza su rol determinante como interfaz intermedia para la verificación semántica de tipos y la generación de código.

Evaluando las divergencias operativas entre las familias de analizadores descendentes (LL, mediante derivación por la izquierda) y ascendentes. Se analizan criterios como la sensibilidad a la recursión izquierda, la complejidad de las tablas de transiciones y su aplicabilidad en lenguajes industriales.

Se explora la función de los "compiladores de compiladores" como Yacc, GNU Bison y ANTLR, detallando sus mecanismos de recuperación de errores para garantizar la resiliencia del analizador ante entradas sintácticamente defectuosas.

Evaluación Experimental de Carga Multitarget (ANTLR4): Se lleva a cabo un experimento cuantitativo sobre la especificación de interfaces de red en archivos Docker Compose. Utilizando ANTLR v4 y su algoritmo $\$ALL(*)\$$, se miden y comparan los tiempos de ejecución de un mismo pipeline sintáctico implementado en Python 3 (interpretado), Java (compilado a bytecode con JIT) y C++ (código nativo) bajo datasets de complejidad lineal ascendente.

Se diseña e implementa un prototipo que combina la rigurosidad determinista de un Parser Descendente Recursivo con un Lexer Híbrido asistido. Este último aplica la métrica de distancia de Levenshtein (ratio de similitud) para corregir errores ortográficos en palabras clave en tiempo de escaneo, permitiendo la construcción fluida del AST sin interrumpir el flujo de compilación.

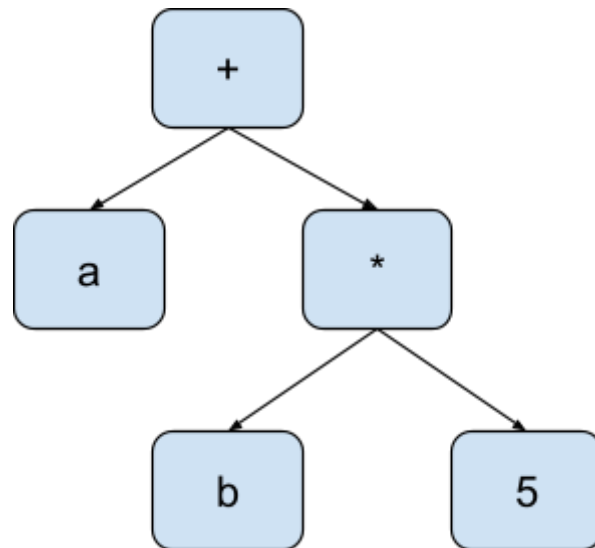
ACTIVIDAD 1: Árbol de sintaxis abstracta (AST)

Un Árbol de Sintaxis Abstracta (AST, por sus siglas en inglés Abstract Syntax Tree) es una representación en forma de árbol n-ario e implícitamente jerárquica de la estructura sintáctica del código fuente de un programa. A diferencia del árbol de sintaxis concreta (o árbol de derivación), el AST omite elementos puramente sintácticos que no aportan significado semántico directo a las fases de optimización o generación de código, tales como paréntesis explícitos, punto y coma de cierre o palabras clave redundantes. Entre sus características principales podemos encontrar:

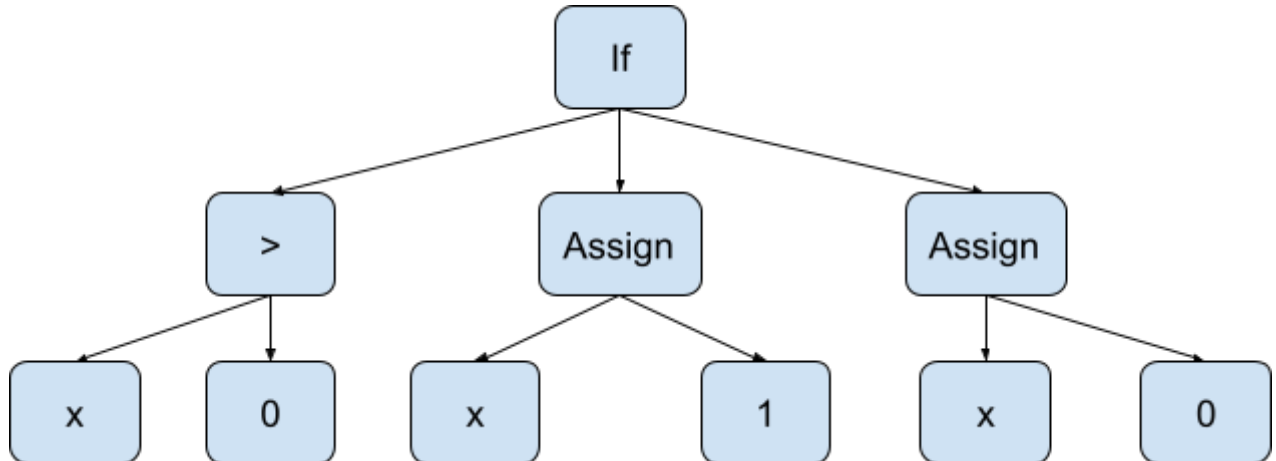
- **Abstracción Sintáctica:** Condensa la información relevante omitiendo símbolos terminales accesorios.
- **Representación Jerárquica:** Los nodos internos representan operadores, instrucciones o constructos sintácticos (ej. asignaciones, bucles, operaciones aritméticas), mientras que los nodos hoja representan operandos o identificadores (variables y literales).
- **Independencia del Formato:** Un mismo AST puede ser producido por gramáticas equivalentes que utilicen distintas notaciones o tokens decorativos.
- **Facilidad de Recorrido:** Sirve como interfaz intermedia estándar para las etapas subsiguientes del compilador, facilitando el análisis semántico (verificación de tipos), las optimizaciones del código intermedio y el recorrido mediante patrones como Visitor.

A modo de ejemplo, podemos tomar el de una expresión aritmética:

Si consideramos la expresión sintáctica de entrada: $A + B * 5$, su representación conceptual jerárquica en forma de AST preserva la precedencia de operadores (“ $*$ ” tiene mayor precedencia que “ $+$ ”), se observa gráficamente de la siguiente forma:



Otro ejemplo sería el de las estructuras condicionales, considerando la sentencia de entrada: `if (x > 0) x = 1; else x = 0;` Su representación conceptual en forma de AST desacopla los paréntesis y palabras clave:



Actividad 2: Comparación entre análisis LL y LR

Los analizadores sintácticos deterministas basados en Gramáticas Libres de Contexto (GLC) se dividen principalmente en dos grandes familias: LL (Left-to-right, Leftmost derivation) y LR (Left-to-right, Rightmost derivation in reverse). A continuación veremos una matriz comparativa en la cual se evalúan los análisis según distintos criterios de comparación:

Criterio de Comparación	Análisis LL	Análisis LR
Dirección de Derivación	Derivación por la izquierda (Leftmost derivation).	Derivación por la derecha en orden inverso (Rightmost derivation in reverse).
Estrategia de Parsing	Descendente (Top-down): Construye el árbol desde la raíz hacia las hojas.	Ascendente (Bottom-up): Reducción desde las hojas hacia la raíz.
Sensibilidad a Recursión	No soporta recursión izquierda directa ni indirecta.	Admite recursión izquierda sin restricciones estructurales.
Complejidad de Tablas	Tablas de análisis compactas e intuitivas.	Tablas explícitas de gran tamaño (estados, desplazamientos y reducciones).
Rango de Gramáticas	Subconjunto más restrictivo ($LL(k) \subset LR(k)$).	Cubre un espectro más amplio de gramáticas de programación.
Facilidad de Implementación	Fácilmente codificable a mano vía Recursive Descent Parsers.	Complejo de programar a mano; se genera mediante

		metacompiladores (Bison, Yacc).
--	--	---------------------------------

En el mundo real, los dos enfoques de análisis sintáctico se usan con propósitos diferentes:

Analizadores LL (Descendentes): Son la opción preferida cuando se busca un excelente manejo de errores y mensajes claros para el desarrollador. Motores como LLVM/Clang utilizan analizadores LL escritos a mano para integrarse de forma fluida con los IDEs, mientras que herramientas como ANTLR usan variantes avanzadas de LL para construir intérpretes, traductores y lenguajes específicos (DSLs).

Analizadores LR / LALR (Ascendentes): Destacan por su velocidad y eficiencia en sintaxis complejas. Se han utilizado históricamente mediante generadores automáticos (como Yacc o Bison) en C, Ruby y OCaml. Además, son el estándar en motores de alto rendimiento para procesar compiladores e intérpretes de GCesar formatos estructurados como JSON y SQL a máxima velocidad.

A modo de ejemplo, si definimos un lenguaje formal L:

$$S \rightarrow id = E;$$

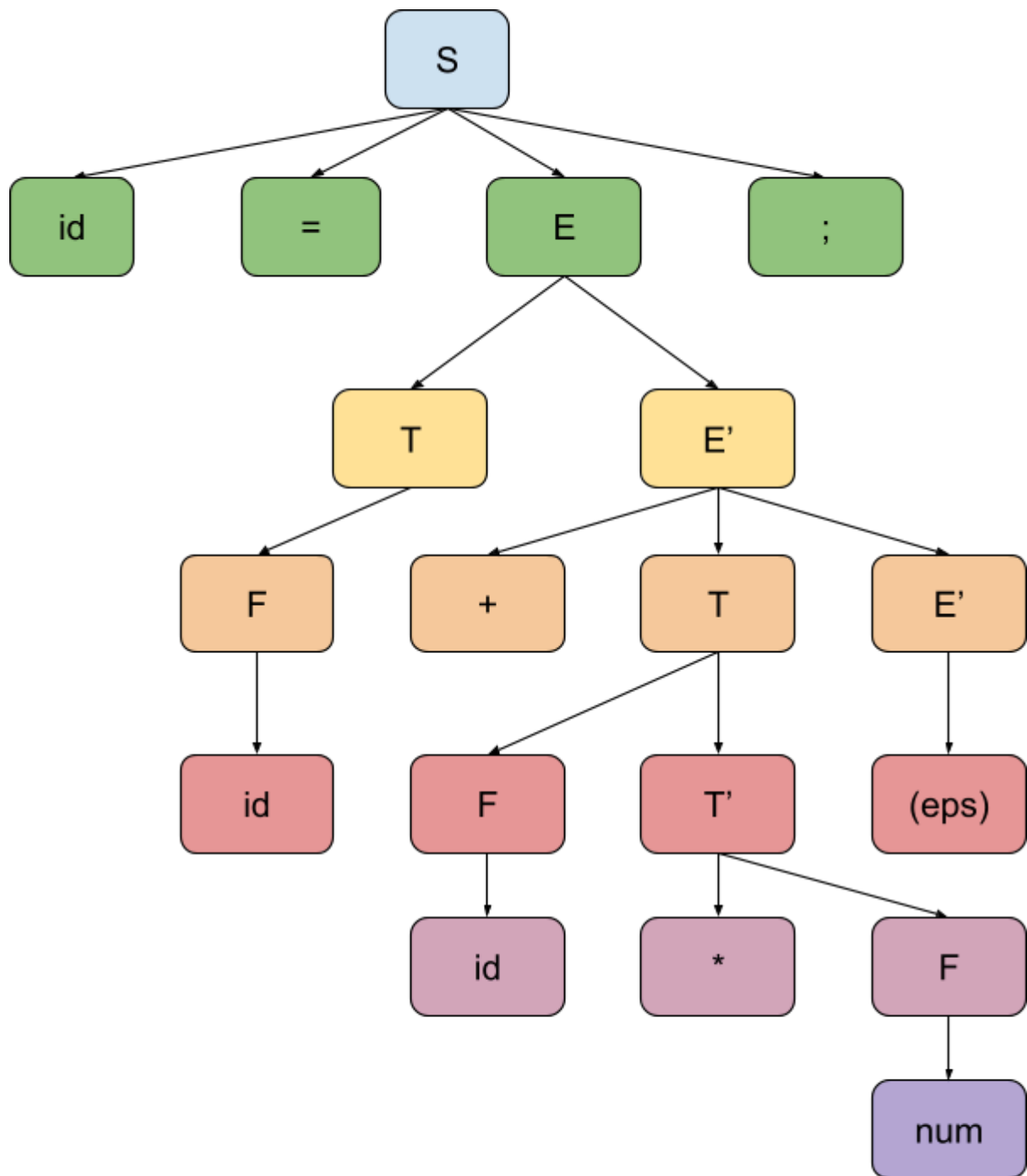
$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow id \mid num$$

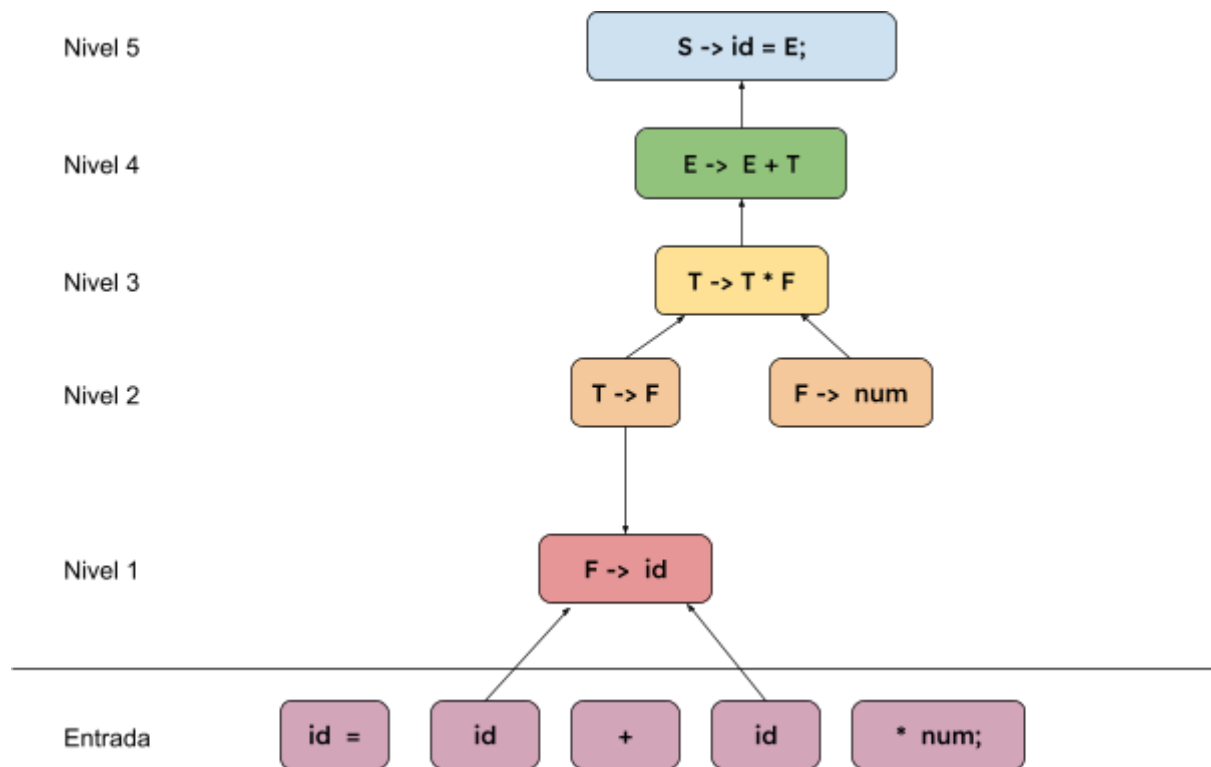
Con una entrada: $id = id + id * num;$

Para el esquema de análisis LL (Top-Down, Derivación por la izquierda):



Eps = Epsilon (no soportado por el modo dibujo)

Para el esquema de análisis LR (Bottom-up, Shift-Reduce):



Generadores de analizadores sintácticos

Un analizador sintáctico constituye el núcleo del proceso de compilación, siendo responsable de transformar una secuencia de tokens, provenientes de la etapa de análisis léxico, en un árbol de derivación o sintaxis abstracta de acuerdo con las reglas matemáticas de una gramática formal. Para automatizar y optimizar la construcción de este crítico módulo, la ingeniería de lenguajes emplea generadores de analizadores sintácticos o "compiladores de compiladores", los cuales toman una especificación gramatical y producen automáticamente el código fuente del analizador en lenguajes como C, C++ o Java. Entre los exponentes históricos e industriales más relevantes destacan Yacc (Yet Another Compiler-Compiler), su iteración de software libre GNU Bison, y modernos marcos de trabajo orientados a objetos como ANTLR (ANother Tool for Language Recognition), herramientas que han consolidado los paradigmas de análisis en la construcción de compiladores comerciales.

Los generadores Yacc/Bison construyen analizadores ascendentes LALR mediante reducción-desplazamiento de hojas a raíz, procesando una amplia clase de gramáticas deterministas. Por el contrario, ANTLR emplea análisis descendente $LL(k)$, prediciendo la

regla aplicable al observar los siguientes k tokens de entrada para crear flujos más legibles. Fundamentalmente, los analizadores LR canónicos garantizan detectar errores antes de realizar desplazamientos inválidos, lo que permite una intervención temprana y precisa.

Los errores informáticos se categorizan en léxicos (caracteres no reconocidos), semánticos y sintácticos; sin embargo, es el manejador de errores del analizador sintáctico el que soporta la mayor carga de la recuperación. Su mandato consiste en informar del fallo con exactitud, sincronizar el flujo de análisis con la suficiente presteza como para localizar irregularidades subsecuentes, y garantizar que este manejo de excepciones no aumente el tiempo de procesamiento de los programas sintácticamente válidos, requiriendo algoritmos que evitan bucles infinitos al procesar sentencias rotas.

Debido a que la infalibilidad del código fuente es una eventualidad excepcional, el componente encargado de la gestión de excepciones en el analizador debe diagnosticar las irregularidades con precisión quirúrgica y restablecer la sincronía del flujo analítico con celeridad. El propósito fundamental de estas maniobras radica en la localización de fallos subsecuentes, garantizando que los algoritmos de recuperación eludan estados de recursión infinita y preserven la eficiencia operativa ante programas sintácticamente íntegros. Para orquestar esta resiliencia, se despliegan diversos paradigmas estratégicos:

Recuperación en Modo Pánico: Representa el mecanismo más extendido en la ingeniería de compiladores, fundamentado en la desestimación secuencial de componentes léxicos hasta converger con un delimitador de sincronización, tal como el punto y coma (;). Si bien asegura la continuidad del proceso, conlleva la omisión de validaciones sobre el segmento de código descartado.

Estrategia a Nivel de Frase: Propone la ejecución de transformaciones locales *in situ* mediante la inserción, supresión o permutación de terminales adyacentes. Esta metodología exige una rigurosidad analítica extrema para prevenir la generación de ciclos lógicos improductivos.

Empleo de Producciones de Error: Consiste en la expansión de la gramática formal mediante reglas auxiliares diseñadas específicamente para capturar patrones de error recurrentes y emitir diagnósticos pormenorizados. No obstante, esta técnica incrementa la complejidad del autómata y la probabilidad de derivar en conflictos de ambigüedad.

Algoritmo de Corrección Global: Determina, mediante el cálculo de la distancia mínima de edición, la secuencia matemáticamente más próxima a una cadena válida. Pese a su rigor teórico, su elevado coste computacional en términos polinomiales restringe su aplicación en entornos de compilación industriales.

La trayectoria histórica en el diseño de compiladores ilustra una clara transición desde los modelos de validación matemáticamente rígidos hacia entornos algorítmicos pragmáticos enfocados en la experiencia del desarrollador y la tolerancia a fallos. Mientras que las utilidades heredadas de los sistemas UNIX, como Yacc, exigen una manipulación empírica de la gramática para lograr sincronizaciones deterministas mediante el truncamiento de la pila de memoria, los marcos orientados a objetos contemporáneos abstraen estas operaciones en clases independientes y manejadores de excepciones predictivos. Independientemente de si el núcleo opera bajo desplazamiento-reducción (LR) o derivación predictiva (LL), el criterio definitivo de un analizador sintáctico industrial no radica meramente en la exhaustividad de los lenguajes que procesa con éxito, sino en la sofisticación analítica con la que logra enmendar e interpretar heurísticamente aquellos que infringen su formalidad.

ACTIVIDAD 4: EXPERIMENTO DE CARGA

Dentro de las fases del proceso de compilación —las cuales abarcan el análisis léxico, análisis sintáctico, análisis semántico, optimización y generación de código—, las primeras dos fases constituyen la base fundamental para el procesamiento y validación de cualquier lenguaje formal o archivo de configuración estructurado.

En la infraestructura de desarrollo moderna, los archivos de especificación como `docker-compose.yml` son utilizados por herramientas de orquestación como Docker para describir de manera declarativa los servicios, volúmenes e interfaces de red de un sistema. Sin embargo, para que un sistema pueda interpretar o validar correctamente las definiciones de red especificadas en estos archivos, es necesario contar con componentes formales capaces de analizar la secuencia de texto de entrada, validar su estructura conforme a una sintaxis predefinida y detectar posibles inconsistencias.

El objetivo abordado en esta problemática consiste en la construcción e integración de un analizador léxico (lexer) y un analizador sintáctico (parser) diseñado específicamente para procesar las especificaciones de interfaz de red en archivos Docker Compose, así como en la evaluación de su rendimiento bajo un experimento de carga comparativo empleando distintas tecnologías e implementaciones de lenguajes de programación.

Análisis Léxico (Lexer) en Archivos de Configuración

El analizador léxico (lexer) constituye la primera fase en el procesamiento del texto fuente. Su función principal es tomar la secuencia o flujo plano de caracteres que conforman el archivo de configuración y convertirla en una secuencia de unidades léxicas con significado denominadas tokens (proceso conocido como tokenización).

En el contexto de los archivos de configuración para interfaces de red en Docker Compose, el lexer cumple las siguientes funciones:

- **Tokenización:** Identifica y clasifica palabras clave (networks, driver, ipam), identificadores de red, valores de parámetros, sangrías o indentaciones y caracteres delimitadores.
- **Abstracción:** Omitir elementos no significativos para la sintaxis (como comentarios y espacios en blanco irrelevantes), simplificando la tarea de la fase sintáctica al abstraer los detalles léxicos del texto fuente.

La implementación del *lexer* se apoya comúnmente en autómatas finitos y expresiones regulares mediante herramientas y metacompiladores como Flex o ANTLR.

Análisis Sintáctico (Parser) y Construcción del AST

El analizador sintáctico (parser) consume de manera secuencial la cadena de tokens generada por el lexer con el objetivo de validar que la estructura del archivo cumpla con las reglas gramaticales establecidas por una Gramática Libre de Contexto (GLC).

Durante el análisis de la configuración de red, el parser evalúa:

- **Validación de Reglas Gramaticales:** Garantiza que las secciones, nombres de red y propiedades sigan la jerarquía sintáctica requerida (por ejemplo, asegurar que un

bloque de definición de red contenga atributos válidos organizados en la sangría correcta).

- **Construcción del Árbol de Sintaxis Abstracta (AST):** Modela la representación jerárquica de la estructura sintáctica del programa o archivo procesado.
- **Detección y Manejo de Errores:** En caso de que el orden o combinación de los tokens viole alguna producción de la GLC, el parser reporta la falla sintáctica y su ubicación.

El parser se puede implementar mediante diversos algoritmos de análisis (como métodos descendentes LL o ascendentes LR/LALR) empleando metacompiladores como ANTLR, Bison o Yacc.

Objetivos del Experimento de Carga

Para evaluar el comportamiento práctico del analizador desarrollado, se plantea un experimento de carga sintáctica empleando metacompiladores sobre un conjunto de 10 archivos

Al generar y ejecutar el lexer-parser en al menos tres (3) lenguajes de programación distintos (como C++, Java y Python), se busca:

1. Medir de forma precisa los tiempos de ejecución consumidos durante la fase de análisis sintáctico.
2. Analizar el impacto que tienen el runtime, las optimizaciones de código y el modelo de ejecución propio de cada lenguaje de programación sobre la eficiencia del parser.
3. Representar gráficamente los tiempos de procesamiento en función de la complejidad o tamaño de los archivos analizados para determinar diferencias de rendimiento.

Metodología y Herramientas Utilizadas

Para el diseño e implementación del analizador léxico y sintáctico de las interfaces de red en archivos Docker Compose, se seleccionó la herramienta metacompiladora **ANTLR v4**. ANTLR utiliza un algoritmo de análisis sintáctico **ALL(*) (Adaptive LL(*))**, lo que **permite procesar gramáticas complejas con búsquedas hacia adelante dinámicas**.

La elección de ANTLR radica en su capacidad multitarget, la cual permite definir una única Gramática Libre de Contexto (GLC) y generar de forma automatizada los analizadores en

múltiples lenguajes de programación objetivo. Para el experimento de carga, se implementaron parsers en tres (3) entornos de ejecución distintos:

1. **Python 3:** Entorno interpretado de alto nivel.
2. **Java (JVM):** Entorno compilado a bytecode con optimizaciones en tiempo de ejecución (JIT).
3. **C++:** Entorno de código objeto nativo y compilación directa a lenguaje máquina.

Generación de Parsers Multitarget

A partir del archivo .g4, se empleó el metacompilador ANTLR4 para compilar la gramática a los lenguajes de destino:

- `antlr4 -Dlanguage=Python3 DockerComposeNetwork.g4`
- `antlr4 -Dlanguage=Java DockerComposeNetwork.g4`
- `antlr4 -Dlanguage=C++ DockerComposeNetwork.g4`

Esto garantiza que la lógica sintáctica evaluada sea exactamente la misma en todos los casos, aislando la variable de prueba a la eficiencia del lenguaje/runtime de ejecución.

Generación del Dataset de Pruebas

Se construyó un dataset de 10 archivos .yaml con complejidad y cantidad de nodos sintácticos escalonada linealmente. Los archivos varían desde infraestructuras pequeñas (10 redes y 20 servicios) hasta infraestructuras complejas (100 redes y 200 servicios).

Estrategia de Medición de Tiempos

Para obtener mediciones precisas e inmunes a interferencias del sistema operativo:

1. Se aisló el temporizador para medir **únicamente** la fase de tokenización y análisis sintáctico (Lexer + Parser), excluyendo tiempos de E/S de disco.
2. Se utilizaron relojes monótonos de alta resolución: `time.perf_counter()` en Python, `System.nanoTime()` en Java y `std::chrono::high_resolution_clock` en C++.
3. Cada archivo fue procesado en 5 repeticiones por cada implementación, calculando el promedio aritmético de los tiempos en milisegundos (ms).

Explicación de cada Programa

El experimento se estructuró mediante la interacción de tres componentes principales:

1 Script Generador de Dataset (generate_dataset.py)

Este programa automatiza la creación del conjunto de pruebas para garantizar condiciones uniformes y reproducibles.

- **Propósito:** Genera de forma paramétrica un conjunto de 10 archivos docker-compose.yml con un nivel de complejidad incremental.
- **Lógica interna:** Utiliza bucles anidados para redactar bloques estructurados de networks y services. El número de redes descritas varía desde 10 redes y 20 servicios en el docker_compose_1.yml, hasta 100 redes y 200 servicios en el docker_compose_10.yml. Esto asegura que cada archivo contenga una carga sintáctica sustancialmente mayor que el anterior.

2. Especificación Formal de Gramática (DockerComposeNetwork.g4)

Es el núcleo del proyecto donde se definen formalmente las reglas del lenguaje que debe reconocer el analizador.

- **Analizador Léxico (Lexer):** Define tokens terminales empleando expresiones regulares para abstraer indentaciones (INDENT), saltos de línea (NEWLINE), identificadores (ID) y valores (VALUE), ignorando elementos irrelevantes como comentarios o espacios en blanco excesivos (skip).
- **Analizador Sintáctico (Parser):** Define la jerarquía de producción basada en una Gramática Libre de Contexto (GLC). La regla principal configFile valida de manera descendente que la sección de redes (networksSection) contenga declaraciones válidas de redes con sus respectivas propiedades (netProperty), y que la sección de servicios (servicesSection) asocie correctamente dichas interfaces de red.

3. Módulos Generados por ANTLR (DockerComposeNetworkLexer.py y DockerComposeNetworkParser.py)

Archivos de código fuente producidos automáticamente por el metacompilador ANTLR4.

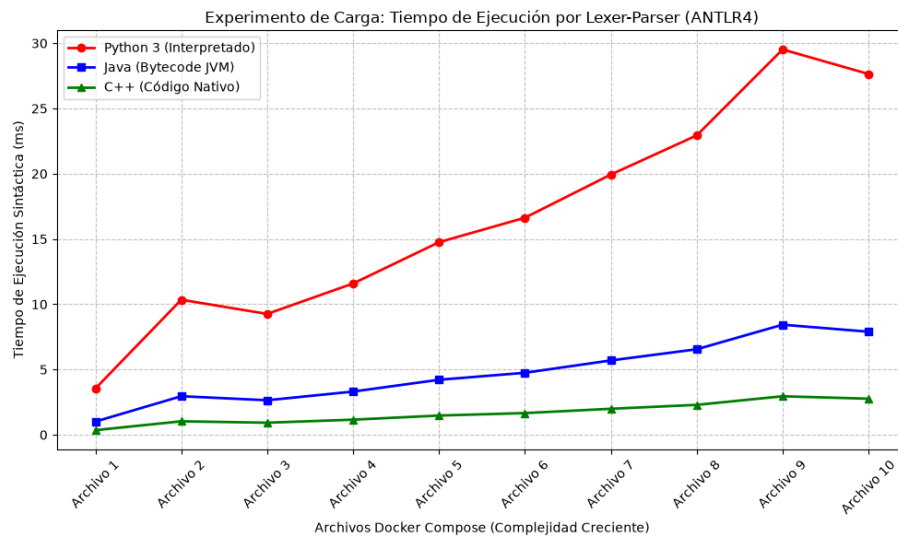
- **DockerComposeNetworkLexer:** Recibe el flujo continuo de caracteres desde el archivo de entrada y los transforma en una secuencia estructurada de objetos Token.
- **DockerComposeNetworkParser:** Consume la secuencia de tokens e intenta construir el Árbol de Sintaxis Abstracta (AST) aplicando el algoritmo de análisis ALL(*).

4. Script de Medición y Automatización (run_benchmark.py)

Es el programa encargado de ejecutar el experimento de carga, medir los tiempos de procesamiento y generar la representación gráfica.

- **Medición Aislada:** Instancia las clases FileStream, Lexer, CommonTokenStream y Parser dentro de un bloque temporal delimitado por time.perf_counter(), asegurando que solo se contabilice el tiempo consumido en el análisis sintáctico.
- **Promediado de Ejecuciones:** Para mitigar variaciones causadas por el sistema operativo, ejecuta el análisis sintáctico de cada archivo 5 veces y calcula el promedio en milisegundos (ms).
- **Generación de Gráficas:** Emplea la librería matplotlib para graficar las curvas comparativas de tiempo en función de la complejidad del archivo.

Discusión de Resultados



En las tres implementaciones evaluadas, el tiempo de ejecución exhibe un comportamiento esencialmente **lineal** con respecto al tamaño y complejidad del archivo analizado. Esto confirma que el analizador sintáctico generado por ANTLR bajo la estrategia de parsing ALL(*) resuelve eficientemente la gramática de interfaces de red de Docker Compose sin incurrir en problemas de retroceso costosos ni ambigüedades sintácticas.

Existe una diferencia drástica en los tiempos de análisis sintáctico entre las tres tecnologías evaluadas, lo que demuestra cómo la arquitectura interna y la optimización del código del entorno de ejecución impactan directamente en la latencia del parser:

1. **Python 3 (Entorno Interpretado):** Presenta la mayor latencia, alcanzando un pico máximo de aproximadamente **29.5 ms** en el archivo 9. Esto se debe a la naturaleza interpretada de CPython, donde la creación, verificación y manipulación dinámica de objetos tipo Token y nodos del árbol en memoria añade un sobrecosto (overhead) significativo por cada regla sintáctica evaluada.
2. **Java (Bytecode & JVM):** Logra un rendimiento significativamente superior, completando el análisis de los archivos más grandes en menos de **8.5 ms** (aprox. 3.5 veces más rápido que Python). Este desempeño responde al compilador Just-In-Time (JIT) de la Máquina Virtual de Java y a la tipificación estática, lo que optimiza la gestión del flujo de tokens y las tablas sintácticas en memoria.

3. **C++ (Código Nativo Compilado):** Ofrece el rendimiento más alto del experimento, con tiempos de respuesta sumamente bajos que apenas superan los **2.8 ms - 3.0 ms** en las pruebas de mayor carga (casi 10 veces más rápido que la versión en Python). Al compilarse directamente a código máquina nativo e interactuar sin la intermediación de una máquina virtual o un intérprete, la asignación de memoria para los nodos del AST y el emparejamiento de expresiones léxicas se ejecutan a velocidad de procesador con un consumo de recursos mínimo.

En la gráfica se observan leves variaciones puntuales (como la pequeña caída de tiempo en el Archivo 10 en comparación con el Archivo 9 en Python y Java). Estas variaciones no corresponden a inconsistencias en la gramática, sino a fluctuaciones normotípicas del entorno de hardware, tales como la planificación de procesos del sistema operativo (CPU scheduling), aciertos en la memoria caché (cache hits) o la recolección de basura (Garbage Collection) en los entornos gestionados.

En conclusión, aunque la definición formal de la gramática sintáctica es idéntica en todos los escenarios, la elección de la tecnología base para construir un compilador o analizador de configuraciones determina de forma crítica la eficiencia del sistema en entornos de alta carga.

ACTIVIDAD 5: ASISTENTE DE PROGRAMACIÓN HÍBRIDO PARA UNEGSCRIPT

El análisis sintáctico tradicional se fundamenta en un esquema estrictamente determinista: ante la presencia de un token no reconocido o una desviación sintáctica mínima, el analizador interrumpe la ejecución o activa rutinas avanzadas de recuperación de errores (como el modo pánico). No obstante, en los entornos de desarrollo modernos se observa una convergencia entre los métodos de compilación clásicos y los mecanismos de asistencia algorítmica.

El objetivo de esta actividad es diseñar e implementar un Asistente de Programación Híbrido para un subconjunto del lenguaje Python denominado *UnegScript*. Este sistema integra un Lexer Híbrido capaz de tolerar errores ortográficos mediante cálculos de similitud léxica y diccionarios de intención (*fallback*), junto con un Parser Recursivo Descendente que construye el Árbol de Sintaxis Abstracta (AST) a partir de los tokens normalizados.

Descripción del Trabajo Realizado

La solución desarrollada aborda el pipeline de compilación mediante un flujo modular articulado en tres fases principales:

1. Fase Léxica (Lexer Híbrido):

- Se definieron los patrones de tokens base mediante Expresiones Regulares (*Regex*) para identificar literales numéricos, cadenas de texto, operadores relacionales, delimitadores e identificadores.
- Para los identificadores que no coinciden exactamente con el conjunto oficial de palabras clave (*KEYWORDS* = {"*print*", "*if*", "*else*"}), el analizador evalúa su distancia de edición frente a las palabras clave válidas.
- Se aplicó la métrica de ratio de coincidencia basada en la distancia de Levenshtein (implementada mediante *difflib.SequenceMatcher*). Si el grado de similitud alcanza o supera el umbral de confianza fijado (≥ 0.8), el sistema corrige el token automáticamente y registra la sugerencia de la IA. En caso contrario, recurre a un diccionario de intenciones como método de fallback.

2. Fase Sintáctica (Parser Recursivo Descendente):

- Se diseñó una jerarquía de clases para modelar los nodos del Árbol de Sintaxis Abstracta (*ProgramNode*, *AssignNode*, *IfNode*, *PrintNode*, *BinOpNode*, *VarNode*, *LiteralNode*). Se implementó un parser descendente manual con capacidad de anticipación de tokens (*lookahead*). El parser analiza la estructura formal de sentencias de asignación, condicionales if-else y llamadas a impresión *print()*, consumiendo la secuencia de tokens y validando la sintaxis.

3. Orquestación y Reporte (*main.py*):

- Se integraron ambos componentes en un módulo principal que recibe el código fuente con errores, procesa la tokenización correctiva, construye el AST jerárquico y despliega un reporte consolidado con las alertas y sugerencias generadas.

Explicación de cada Programa

a. Analizador Léxico Híbrido ([lexer.py](#))

Este programa contiene la función `lexer_hibrido(codigo, umbral=0.8)`, responsable de la tokenización primaria y la corrección ortográfica.

- ❖ *Procesamiento RegEx*: La especificación `TOKEN_SPEC` unifica las expresiones regulares mediante grupos nombrados.
- ❖ *Cálculo de Similitud Léxica*: Cuando se encuentra un identificador desconocido, se calcula la razón de edición frente a las palabras clave oficiales:

$$ratio = 1 - \{Distancia\ de\ Levenshtein(s1, s2)\} / \{max(len(s1), len(s2))\}$$

En Python, este cálculo se realiza mediante `SequenceMatcher(None, lexeme, candidato).ratio()`.

Corrección y Fallback: Si $ratio \geq 0.8$, el token se transforma a su respectivo `KW_<KEYWORD>`. Si cae por debajo del umbral, se consulta el diccionario de fallback (`{"pront": "print", "prnt": "print"}`) antes de declararlo como una variable ordinaria (`IDENTIFIER`).

```
# Fragmento del cálculo de similitud en lexer.py

for kw in KEYWORDS:
    ratio = SequenceMatcher(None, value, kw).ratio()
    if ratio > max_ratio:
        max_ratio = ratio
        mejor_match = kw

if max_ratio >= umbral:
    sugerencias_ia.append(f"Sugerencia: '{value}' → '{mejor_match}' (Confianza: {max_ratio:.2f})")
    tokens.append(("KW_" + mejor_match.upper(), mejor_match))
```

b. Analizador Sintáctico y AST ([parser_unegscript.py](#))

Este módulo implementa el parser recursivo descendente a través de la clase `ParserUnegScript`.

- *Métodos de Control*: peek(offset) permite inspeccionar tokens sin avanzar la posición actual, mientras que consume(expected_type) valida y avanza en la secuencia léxica.
- *Construcción de Producciones*:
 - ❖ **parse_if()** reconoce la estructura del condicional evaluando la expresión lógica y construyendo las ramas then_branch y else_branch.
 - ❖ **parse_print()** soporta invocaciones tanto con paréntesis como directas.
 - ❖ **parse_expression()** resuelve operaciones relacionales y asignaciones dentro de las sentencias.
- *Visualización*: La función mostrar_ast(node, indent) recorre el árbol de forma recursiva e imprime la estructura jerárquica con sangrías proporcionales a la profundidad del nodo.

c. Script de Integración y Pruebas ([main.py](#))

Constituye el punto de entrada principal del asistente. Coordina la ejecución secuencial enviando el código fuente al lexer, transfiriendo los tokens corregidos al parser y formateando las tres salidas requeridas.

Resultados de Ejecución

Se evaluó la implementación utilizando el caso de prueba estipulado en la guía práctica:

Código de entrada: `pront x=5; if x>3 prnt(x) else prnt("no")`

Al ejecutar python main.py, la consola arrojó la siguiente salida:

```
=====
ASISTENTE DE PROGRAMACIÓN HÍBRIDO - UNEGSCRIPT
=====

CÓDIGO DE ENTRADA:
pront x=5; if x>3 prnt(x) else prnt("no")

-----
1. TOKENS CORREGIDOS
-----
```

```
[KW_PRINT ] -> 'print'
[IDENTIFIER ] -> 'x'
[ASSIGN ] -> '='
[NUMBER ] -> '5'
[SEMI ] -> ';'
[KW_IF ] -> 'if'
[IDENTIFIER ] -> 'x'
[OP_REL ] -> '>'
[NUMBER ] -> '3'
[KW_PRINT ] -> 'print'
[LPAREN ] -> '('
[IDENTIFIER ] -> 'x'
[RPAREN ] -> ')'
[KW_ELSE ] -> 'else'
[KW_PRINT ] -> 'print'
[LPAREN ] -> '('
[STRING ] -> '"no"'
[RPAREN ] -> ')'
```

2. ÁRBOL DE SINTAXIS ABSTRACTA (AST)

ProgramNode:

PrintNode:

AssignNode (var='x'):

LiteralNode (val=5, type=int)

IfNode:

Condición:

BinOpNode (op='>'):

VarNode ('x')

LiteralNode (val=3, type=int)

Rama THEN:

PrintNode:

VarNode ('x')

Rama ELSE:

PrintNode:

LiteralNode (val="no", type=str)

3. SUGERENCIAS DE IA / REPORTE DE CORRECCIONES

- Sugerencia: 'pront' → 'print' (Confianza: 0.80)
 - Sugerencia: 'prnt' → 'print' (Confianza: 0.89)
 - Sugerencia: 'prnt' → 'print' (Confianza: 0.89)
-

Discusión de Resultados

1. Efectividad del Umbral Léxico (≥ 0.8):

El cálculo del ratio de similitud demostró ser preciso ante variaciones ortográficas comunes: El lexema 'pront' obtuvo una coincidencia de 0.80 frente a 'print' (4 de 5 caracteres coincidentes en posición).

- El lexema '*prnt*' obtuvo un índice de confianza de 0.89 frente a 'print'.
- En ambos casos, al superar el umbral fijado de 0.8, el sistema corrigió los tokens de forma transparente antes de ser enviados al parser, evitando que el analizador sintáctico colapsara por errores léxicos.

2. Robustez en la Construcción del AST:

Gracias a la normalización léxica previa, el parser recursivo descendente procesó la gramática sin interrupciones. El AST resultante separó correctamente las instrucciones:

- Convirtió la primera sentencia en una asignación integrada dentro del flujo principal.
- Evaluó la expresión relacional $x > 3$ como un nodo binario *BinOpNode* asignado a la condición del *IfNode*.
- Desacopló los delimitadores y paréntesis sintácticos, manteniendo en las ramas THEN y ELSE únicamente la semántica de las llamadas a *PrintNode*.

3. Determinismo vs. Asistencia Algorítmica:

El experimento valida la eficacia de una arquitectura híbrida. Mientras que un compilador tradicional habría rechazado la entrada por tokens desconocidos (*pront/prnt*), la capa de corrección por similitud actuó como un filtro tolerante a fallas, garantizando que las fases posteriores del compilador (análisis sintáctico y generación del AST) operen sobre estructuras sintácticamente válidas.

Conclusión

El desarrollo de las actividades presentadas en este informe permite reafirmar que el análisis sintáctico no es una fase aislada, sino el eje organizador de la semántica de cualquier procesador de lenguaje o analizador de configuraciones.

A través del estudio de los Árboles de Sintaxis Abstracta (AST) y la comparativa entre los paradigmas LL y LR, se evidencia cómo la abstracción de elementos accesorios (como paréntesis o delimitadores) simplifica las fases posteriores de optimización y recorrido mediante patrones de diseño. Mientras que los analizadores descendentes (LL) destacan por su legibilidad y diagnósticos claros de error en entornos de desarrollo, los enfoques ascendentes (LR/LALR) sobresalen en potencia expresiva y rendimiento para gramáticas complejas.

El experimento de carga implementado sobre especificaciones de red en Docker Compose mediante ANTLR4 demostró que, aunque la lógica gramatical expresada formalmente ($\$ALL(*)\$$) sea matemáticamente equivalente en todas las plataformas, el entorno de ejecución (runtime) condiciona de forma drástica la eficiencia:

- **C++ (Código Nativo):** Ofrece el rendimiento óptimo (latencias de ~2.8 ms - 3.0 ms en escenarios de alta carga), gracias a la asignación directa de memoria a nivel de procesador.
- **Java (JVM & JIT):** Alcanza un equilibrio de alto desempeño (~8.5 ms), beneficiándose de las optimizaciones dinámicas en tiempo de ejecución de la máquina virtual.
- **Python 3 (Interpretado):** Exhibe la mayor latencia (~29.5 ms), reflejando el sobrecosto (*overhead*) derivado del tipado dinámico y la instanciación de objetos en memoria por parte del intérprete CPython.

Finalmente, la integración del Asistente Híbrido para UnegScript validó la viabilidad de converger la teoría clásica de compiladores con técnicas tolerantes a fallos basadas en similitud léxica (Levenshtein). La capacidad de normalizar automáticamente errores ortográficos comunes (como `pront` o `prnt` hacia `print`) manteniendo un umbral de confianza igual o superior a 0.80 permitió al parser construir el AST de forma continua y determinista. Este enfoque demuestra que es posible diseñar arquitecturas de compilación resilientes capaces de guiar al desarrollador sin colapsar ante desviaciones menores en la entrada.

Referencias bibliográficas

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2.^a ed.). Pearson Educación.
- Alonso, L. (s.f.). *Procesadores de lenguaje: Compiladores*. Universidad de Salamanca. <http://avellano.fis.usal.es/~lalonso/procesadores/compiladores.pdf>
- Maruri Parada, E. Y. (2024). *Compendio sobre Flex y Bison*. Instituto Politécnico Nacional. <https://es.scribd.com/document/785727315/flex-y-bison>
- Parr, T. (2013). *The Definitive ANTLR 4 Reference* (2nd ed.). Pragmatic Bookshelf.
- Red Iberoamericana de Academias de Investigación. (2020). *Vanguardia tecnológica, industria 4.0 y sistemas inteligentes*. REDIBAI. <https://redibai-myd.org/portal/wp-content/uploads/2020/01/8617-41-8.pdf>